A DATA SCIENCE LAB REPORT

ON

WEB SCRAPING AND DATASET PREPARATION

COURSE CODE: CSE 4114

COURSE TITLE: Introduction To Data Science Lab

NORTHERN UNIVERSITY OF BUSINESS & TECHNOLOGY

KHULNA

DEPARTMENT OF CSE

Submitted By:	Submitted To:
Name: Sajal Biswas ID: 11230121114 Section: 7B Department: CSE NORTHERN UNIVERSITY OF BUSINESS & TECHNOLOGY, KHULNA	Name: Vashkar Kar Lecturer, Department of CSE NORTHERN UNIVERSITY OF BUSINESS & TECHNOLOGY, KHULNA

Submission Date: 20-08-26 Signature

CVE Vulnerability Dataset Collection and Analysis

1. **Introduction**

Cybersecurity vulnerabilities are security weaknesses in software, libraries, frameworks, and applications that may be exploited by attackers. The Common Vulnerabilities and Exposures (CVE) system provides unique identifiers for publicly known vulnerabilities.

This project focuses on collecting cybersecurity vulnerability data from the **Open Source Vulnerabilities (OSV)** database and preparing it as a structured CSV dataset. The project uses **Python and BeautifulSoup** for web scraping and data extraction. The target is to collect approximately **1500+ usable CVE records** and prepare the dataset for further data analysis.

2. Domain Description

The selected domain for this project is **Cybersecurity Vulnerability Analysis**.

The project specifically focuses on CVE-related information, including vulnerability severity, CVSS scores, affected software packages, vulnerability descriptions, CWE classifications, affected versions or commits, publication dates, and references.

Analyzing this information can help identify patterns such as:

- Which vulnerabilities have the highest severity.
- Which software ecosystems are most represented.
- Which types of weaknesses occur most frequently.
- How vulnerability severity changes over time.
- Which software packages are frequently affected.

3. Objectives

The main objectives of this project are:

1. Collect approximately **1500+ CVE records** from OSV.
2. Use **BeautifulSoup** for HTML parsing and data extraction.
3. Store the collected information in a structured CSV file.
4. Clean and organize the collected dataset.
5. Perform exploratory data analysis on the vulnerability data.
6. Identify trends and patterns in cybersecurity vulnerabilities.
7. Upload the source code and dataset to GitHub for reproducibility.

4. Data Source

The primary data source for this project is the **Open Source Vulnerabilities (OSV) database**.

OSV Website: Open Source Vulnerabilities (OSV)

OSV provides publicly available vulnerability information from multiple sources. Individual CVE records contain information such as CVE identifiers, descriptions, severity, CVSS information, affected packages, version or commit ranges, CWE identifiers, and references.

The project collects information from OSV vulnerability pages using HTTP requests and **BeautifulSoup** for HTML parsing.

5. Dataset Description

The collected dataset is stored in CSV format. The main attributes are:

Attribute	Type	Description
cve_id	String	Unique CVE identifier
source	String	Original CVE source URL
import_source	String	OSV import source
json_data	String	OSV JSON data reference
aliases	String	Other identifiers associated with the vulnerability
published	DateTime	Date when the vulnerability was published
modified	DateTime	Date when the vulnerability information was last modified
severity	String	Vulnerability severity and rating
cvss_score	Float	CVSS numerical score
cvss_version	String	CVSS version used
cvss_vector	String	CVSS vector describing vulnerability characteristics
summary	String	Short vulnerability description
details	Text	Detailed vulnerability description
ecosystem	String	Software ecosystem
package	String	Affected software package
range_type	String	Type of affected version/range information

Attribute	Type	Description
repo	String	Related source-code repository
introduced	String	Vulnerability introduction version or commit
fixed	String	Version or commit where the vulnerability was fixed
cwe_ids	String	CWE classification
cna_assigner	String	Organization responsible for assigning the CVE
references	Text	External references related to the vulnerability

---

6. Technologies and Libraries Used

The following technologies and libraries are used:

Programming Language

- **Python 3**

Libraries

- **Requests** – Used to send HTTP requests to OSV.
- **BeautifulSoup4** – Used to parse HTML and extract vulnerability information.
- **Pandas** – Used for CSV creation, data manipulation, and analysis.
- **CSV** – Used for structured dataset storage.
- **Time/Random** – Used to introduce delays between requests and reduce excessive request frequency.

Development Environment

- Linux Terminal
- Python Virtual Environment (venv)
- Git
- GitHub

7. Methodology

The project follows a structured data collection process.

Step 1: Identify the Data Source

OSV was selected as the primary vulnerability data source because it provides structured and publicly accessible vulnerability information.

Step 2: Identify CVE IDs

A list of CVE identifiers was prepared for the selected time period. A larger list was collected than the final target because some CVEs may not have corresponding normal OSV vulnerability pages.

Step 3: Request OSV Pages

Python's requests library is used to request individual OSV vulnerability pages.

Example: https://osv.dev/vulnerability/CVE-2024-3094

Step 4: Parse HTML

BeautifulSoup is used to analyze the returned HTML document and locate relevant elements such as:

- CVE ID
- Published date
- Modified date
- Severity
- CVSS score
- Summary
- Details
- Affected package
- Repository
- CWE
- References

Step 5: Extract Data

The extracted values are converted into structured Python data.

Step 6: Store Data

Each successfully processed vulnerability is stored as a row in:

data/osv_cve_dataset.csv

Step 7: Handle Missing Records

Some CVE IDs may not have a standard vulnerability page on OSV. These records are skipped or marked as unsuccessful rather than being treated as valid dataset records.

Step 8: Handle Network Problems

Because thousands of pages are being requested, temporary errors such as HTTP 429, DNS failures, or connection errors may occur. Delays and retry mechanisms are used to make the collection process more reliable.

Step 9: Data Cleaning and Analysis

After collection, the CSV dataset will be cleaned and analyzed using Pandas and other data-analysis techniques.

8. Data Cleaning

The raw scraped data requires cleaning before analysis.

The following activities will be performed:

1. **Remove duplicate CVE IDs**:Each CVE should appear only once in the final dataset.
2. **Handle missing values**:Some vulnerabilities may not contain CVSS scores, CWE IDs, affected versions, or other attributes.
3. **Normalize severity values**:Severity values will be standardized into categories such as:
    - Critical
    - High
    - Medium
    - Low
4. **Convert dates**:published and modified values will be converted into appropriate datetime formats.
5. **Convert CVSS scores**:CVSS values will be stored as numerical values to support statistical analysis.
6. **Clean text fields**:Unnecessary whitespace, duplicated spaces, and HTML-related artifacts will be removed.
7. **Normalize categorical fields**:Ecosystem and CWE values will be standardized where necessary.
8. **Validate records**:CVE identifiers and important fields will be checked to ensure that the collected records contain valid information.

9. Dataset Quality and Limitations

Although OSV provides a valuable vulnerability dataset, several limitations exist.

Dataset Quality

The dataset contains information from a recognized vulnerability database and includes multiple useful attributes such as CVSS, CWE, affected packages, and vulnerability descriptions.

The use of automated extraction also makes the collection process reproducible.

Limitations

- Not every CVE has a normal vulnerability page on OSV.
- Some records may have missing attributes.
- Different vulnerability records may contain different levels of detail.
- Some CVSS information may use different CVSS versions.
- Affected software may be represented using versions, commits, or ranges.
- OSV data may be updated after the dataset is collected.
- Temporary network errors or HTTP rate limiting can cause some records to fail during scraping.
- The dataset represents the selected collection period and therefore may not represent all existing vulnerabilities.

Because of these limitations, the final dataset should be considered a **collected research dataset rather than a complete representation of every CVE**.

10. Possible Data Science Analysis

The collected dataset can be used for several data-analysis tasks.

Severity Analysis

Analyze the distribution of vulnerabilities by severity:

Critical

High

Medium

Low

CVSS Analysis

Calculate:

- Average CVSS score
- Minimum CVSS score
- Maximum CVSS score
- CVSS score distribution

Time-Series Analysis

Analyze the number of vulnerabilities published per year or month.

CWE Analysis

Identify the most frequently occurring weakness categories.

For example:

CWE-79 → Cross-Site Scripting

CWE-89 → SQL Injection

CWE-787 → Out-of-Bounds Write

Ecosystem Analysis

Determine which ecosystems occur most frequently in the dataset, such as:

- Git
- PyPI
- npm
- Maven
- Go
- RubyGems
- crates.io

Package Analysis

Identify software packages that appear frequently among the collected vulnerabilities.

Correlation Analysis

Investigate relationships between:

- CVSS score and severity
- Publication year and severity
- CWE and CVSS score
- Ecosystem and vulnerability severity

Visualization

Possible visualizations include:

- Bar charts
- Pie charts
- Histograms
- Time-series plots
- Heatmaps
- CWE frequency charts

11. Expected Outcome

The expected outcome is a structured dataset containing approximately **2,000 valid CVE records** collected from OSV.

The final project should contain:

```
Data-Science-Lab/
├─── data/
│    ├─── cve_ids.txt
│    └─── osv_cve_dataset.csv
├─── src/
│    ├─── get_cve_ids.py
│    ├─── collect_cves.py
├─── README.md
└─── requirements.txt
```

The project will provide a reusable cybersecurity dataset that can be analyzed using Python and Pandas.

The results may help identify common vulnerability patterns, severity trends, affected ecosystems, and frequently occurring weakness categories.

12. Conclusion

This project demonstrates how web scraping and data science techniques can be applied to cybersecurity vulnerability information. By using **Python, Requests, and BeautifulSoup**, vulnerability information can be collected from OSV and transformed into a structured CSV dataset.

The collected dataset can then be cleaned and analyzed using Pandas to discover meaningful cybersecurity trends. The project also demonstrates practical challenges associated with large-scale web scraping, including missing records, inconsistent data, network failures, and rate limiting.

Finally, publishing the source code and dataset on GitHub will make the project reproducible and provide a useful resource for further cybersecurity and data-science research.

13. References

1. https://osv.dev/?utm_source=chatgpt.com
2. https://ossf.github.io/osv-schema/?utm_source=chatgpt.com
3. https://www.cve.org/?utm_source=chatgpt.com
4. https://cwe.mitre.org/?utm_source=chatgpt.com
5. https://www.crummy.com/software/BeautifulSoup/bs4/doc/?utm_source=chatgpt.com
6. https://pandas.pydata.org/docs/?utm_source=chatgpt.com
7. https://nvd.nist.gov/?utm_source=chatgpt.com